

This is a **production-level topic** — if you explain this well, you sound like someone who can **run and maintain real Node.js services**, not just write code 🔥

Let's go deep with **clear explanations + practical code you can speak through confidently**.

5) Process Management in Node.js

1. process.pid

What it is:

Every Node.js process has a unique **Process ID (PID)** assigned by the OS.

Interview answer:

“process.pid returns the unique identifier of the running Node.js process, which is useful for logging, debugging, and managing processes.”

Example:

```
JavaScriptconsole.log("Process ID:", process.pid);
```

👉 Useful in:

- Logs
 - Debugging clustered apps
 - Monitoring tools
-

2. process.env

What it is:

Access to **environment variables**

Interview answer:

“process.env is used to store configuration values like API keys, database URLs, and environment modes, keeping them separate from code.”

Example:

```
JavaScriptconsole.log(process.env.NODE_ENV);  
console.log(process.env.PORT);
```

Real-world usage:

```
JavaScriptconstPORT=process.env.PORT || 3000;
```

Why important:

- Security (no secrets in code)
 - Config per environment (dev, prod)
-

3. process.memoryUsage()

What it is:

Gives memory usage details of your app

Interview answer:

“process.memoryUsage() returns memory consumption details like heap and RSS, which helps in performance monitoring and detecting memory leaks.”

Example:

```
JavaScriptconsole.log(process.memoryUsage());
```

Output example:

```
JavaScript{  
  rss: 23429120,  
  heapTotal: 4907008,  
  heapUsed: 2905912,  
  external: 951886  
}
```

Key fields:

- rss → total memory used
 - heapUsed → actual JS memory
-

4. process.exit()

What it is:

Terminates the process

Interview answer:

“process.exit() is used to terminate the Node.js process with an exit code indicating success or failure.”

Example:

```
JavaScriptconsole.log("Something went wrong");  
process.exit(1);
```

Exit codes:

- 0 → success
 - 1 → error
-

⚠ Important:

Avoid calling `process.exit()` abruptly in production — it can kill ongoing requests.

5. Handling Uncaught Exceptions

Problem:

If an exception is not caught → app crashes

Example:

```
JavaScriptsetTimeout(() => {  
  throw new Error("Crash!");  
}, 1000);
```

Solution:

```
JavaScriptprocess.on("uncaughtException", (err) => {  
  console.error("Uncaught Exception:", err.message);
```

```
// Cleanup + exit  
process.exit(1);  
});
```

Interview answer:

“We can handle uncaught exceptions using `process.on('uncaughtException')`, log the error, perform cleanup, and exit safely.”

⚠ Best practice:

- Don't continue running → app state may be corrupted
-

6. Handling Unhandled Promise Rejections

Problem:

A rejected promise without `.catch()` can crash or behave unpredictably

Example:

```
JavaScriptPromise.reject("Something failed");
```

Solution:

```
JavaScriptprocess.on("unhandledRejection", (reason, promise) => {  
  console.error("Unhandled Rejection:", reason);
```

```
  process.exit(1);  
});
```

Interview answer:

“Unhandled promise rejections can be caught using `process.on('unhandledRejection')` to log and safely terminate the process.”

7. Graceful Shutdown (SIGTERM / SIGINT)

🔥 **VERY IMPORTANT for interviews (production readiness)**

What is it?

When your app is told to shut down:

- SIGINT → Ctrl+C
 - SIGTERM → from Docker, Kubernetes, PM2, etc.
-

Problem:

If you exit immediately:

- Requests may be lost ❌
 - DB connections left open ❌
-

Proper solution: Graceful shutdown

Example:

```
JavaScriptconst http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(3000, () => {
  console.log("Server running on port 3000");
});

function shutdown(signal) {
  console.log(`Received ${signal}. Shutting down...`);

  server.close(() => {
```

```
console.log("All connections closed");
process.exit(0);
});
```

```
// Force shutdown after timeout
setTimeout(() => {
  console.error("Force shutdown");
  process.exit(1);
}, 5000);
}
```

```
process.on("SIGINT", shutdown);
process.on("SIGTERM", shutdown);
```

What this does:

- Stops accepting new requests
 - Finishes existing requests
 - Closes server cleanly
-

Interview answer:

“Graceful shutdown ensures that the server stops accepting new requests, completes ongoing ones, and releases resources before exiting, typically handled using SIGINT and SIGTERM signals.”

Full Production Pattern

```
JavaScriptprocess.on("uncaughtException", (err) => {
  console.error("Uncaught Exception:", err);
  process.exit(1);
});
```

```
process.on("unhandledRejection", (err) => {
  console.error("Unhandled Rejection:", err);
  process.exit(1);
});
```

```
process.on("SIGTERM", shutdown);
process.on("SIGINT", shutdown);

function shutdown() {
  console.log("Shutting down gracefully...");
  // close DB, server, queues
  process.exit(0);
}
```

Strong Interview Explanation (Use This)

“Process management in Node.js involves monitoring and controlling the lifecycle of the application. We use `process.pid` for identification, `process.env` for configuration, and `process.memoryUsage()` for performance monitoring. Errors like uncaught exceptions and unhandled promise rejections must be handled to prevent crashes. In production, we implement graceful shutdown using `SIGINT` and `SIGTERM` to safely close connections and avoid data loss.”

Quick Revision

- `process.pid` → process ID
- `process.env` → environment config
- `process.memoryUsage()` → memory tracking
- `process.exit()` → terminate process
- Handle:
 - `uncaughtException`
 - `unhandledRejection`
- Graceful shutdown:
 - `SIGINT`, `SIGTERM`

- o close server before exit
-

If you want next-level prep 🔥

I can:

- Combine **Cluster + Worker Threads + Process mgmt** into one architecture answer
- Give real interview questions (like FAANG-style follow-ups)

- Or
next topic

Just send it 🚀

continue with